

# ToddySoft Connect ctrlX Driver - Ignition 8.3

---

Ignition 8.3 module for connecting to Bosch Rexroth **ctrlX CORE** controllers over the ctrlX Data Layer's HTTPS REST API. Every node the Data Layer exposes is reachable the same way — PLC symbols, axis state, framework metrics — addressed by its slash-separated node path.

The device configuration uses a custom configuration form: settings that do not apply to what you have selected stay out of your way, and the fields that carry a security consequence say so where you read them.

**This is an HTTPS management interface, not a fieldbus.** Each tag in a read costs one HTTP request, so this driver suits diagnostics, configuration and moderate-rate supervision rather than high-rate process data. Size your tag groups accordingly; see *Tag Groups*.

## Installation

---

**This module is for Ignition 8.3 only.** It is built against the Ignition 8.3 SDK and will not run on any other major version. For Ignition 8.1, install the separate `...-8.1.mod1` build instead — the two are not interchangeable, and each gateway needs the build that matches it.

**Ignition 8.0 and earlier are not supported and cannot be.** The module is built on the device-driver API that Ignition introduced with 8.1, so on 8.0 the gateway fails to load it with `NoClassDefFoundError: ... opcua.server.api.AbstractDeviceModuleHook`.

Both builds carry the same name in the gateway's module list, so tell them apart by the file you installed or by the module description, which names the Ignition version.

1. Open the Ignition Gateway web interface and navigate to **Config > System > Modules**.
2. Click **Install or Upgrade a Module...**

3. Select the `ToddySoft-Connect-CtrlX-Driver-8.3.modl` file and upload it.
4. Accept the license / certificate prompt to continue the installation.
5. Wait for the Gateway to finish and verify the module appears as **Running** in the modules list.

## Before you start: the controller side

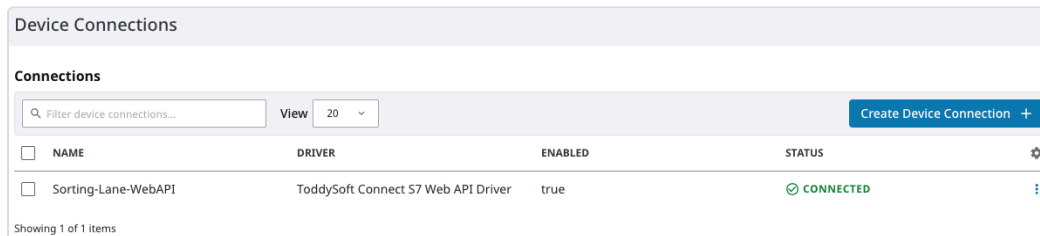
Two things must be true on the ctrlX CORE before a connection can succeed.

- **An account with the right role.** ctrlX manages read and write as separate permissions, so an account that browses and reads happily may still refuse every write with HTTP 403. Roles are assigned in the controller's own user management.
- **The Data Layer service running**, and HTTPS reachable on port 443 (or whatever you override the port to).

The certificate a ctrlX CORE ships with is self-signed and chains to no public CA, so ordinary verification cannot pass. See *Reaching a device with its factory certificate* below — the answer is to pin it, not to switch verification off and leave it off.

## Creating a Device Connection

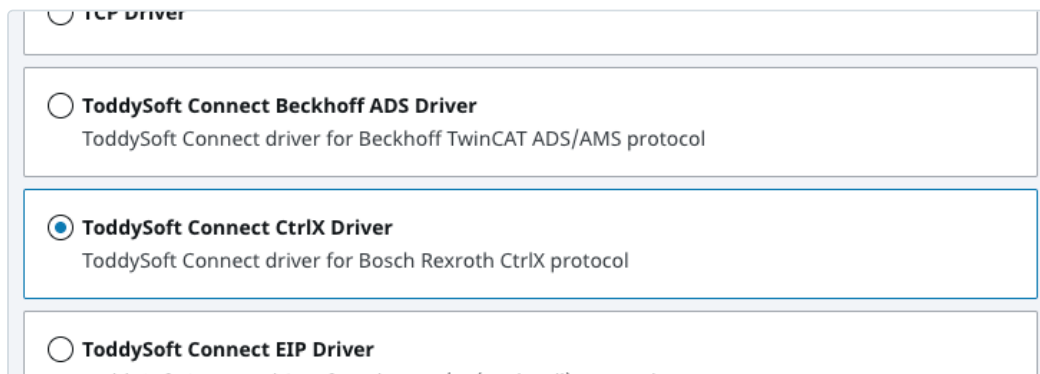
1. Go to **Config > OPC UA > Device Connections**.



| Device Connections           |                     |                                     |         |                            |
|------------------------------|---------------------|-------------------------------------|---------|----------------------------|
| Connections                  |                     |                                     |         |                            |
| Filter device connections... |                     |                                     | View 20 | Create Device Connection + |
| <input type="checkbox"/>     | NAME                | DRIVER                              | ENABLED | STATUS                     |
| <input type="checkbox"/>     | Sorting-Lane-WebAPI | ToddySoft Connect S7 Web API Driver | true    | CONNECTED                  |

Showing 1 of 1 items

2. Click **Create new Device...** and select **ToddySoft Connect CtrlX Driver** as the device type.



☐ TCP Driver

☐ ToddySoft Connect Beckhoff ADS Driver  
ToddySoft Connect driver for Beckhoff TwinCAT ADS/AMS protocol

☒ **ToddySoft Connect CtrlX Driver**  
ToddySoft Connect driver for Bosch Rexroth CtrlX protocol

☐ ToddySoft Connect EIP Driver

3. Configure the connection settings (see the sections below) and click **Create Device Connection**.
4. The new device then appears in the Device Connections list.

| Device Connections                                        |                     |                                     |                                             |           |
|-----------------------------------------------------------|---------------------|-------------------------------------|---------------------------------------------|-----------|
| Connections                                               |                     |                                     |                                             |           |
| <input type="text" value="Filter device connections..."/> |                     | View 20                             | <button>Create Device Connection +</button> |           |
| <input type="checkbox"/>                                  | NAME                | DRIVER                              | ENABLED                                     | STATUS    |
| <input type="checkbox"/>                                  | CtrlX-Bench         | ToddySoft Connect CtrlX Driver      | true                                        | CONNECTED |
| <input type="checkbox"/>                                  | Sorting-Lane-WebAPI | ToddySoft Connect S7 Web API Driver | true                                        | CONNECTED |

**Ignition 8.3 can find ctrlX devices for you; Ignition 8.1 cannot.** The 8.1 device page has no discovery action for any driver — enter the host by hand there. Neither version offers a *Test Connection* action for ctrlX: create the device and read its status.

## Finding devices

Above the configuration form sits **Discover Devices**. It searches the gateway's own networks and lists the ctrlX controllers that answer.

Create Device Connection

Create Device Connection

1. Select Driver

2. Configure Device

Configure Device

GENERAL

Name\*

A unique name for this device connection.

Description

An optional description for this device connection.

Enabled

☒ Set whether this device connection is enabled or disabled within the Gateway.

Discover Devices

TRANSPORT

Host

Hostname or IP address of the CtrlX device

Port

Back

Create Device Connection

A ctrlX announces itself over mDNS, but nothing in that announcement identifies it as a ctrlX: the search asks the generic question every web-serving device on the network answers, and both the name and the hostname a ctrlX advertises are chosen by whoever commissioned it. So each responder is then asked one unauthenticated question that only a ctrlX Data Layer answers the way a ctrlX does, and only the ones that confirm are listed. That is why the search takes a few seconds per network interface — and why a controller you renamed is still found.

Discover Devices

Devices that answered a broadcast on this gateway's networks.

Timeout (ms)

25000

Rescan

How long to wait for devices to answer. The gateway clamps this to its own bounds.

ICX Launcher Site

192.168.42.100

rest

Hostname ctrlX-OS.local

Cancel

Apply

Selecting one shows what that device reported — the connection URL it would be reached on, its hostname and its addresses — before anything is written into the form.

Discover Devices

Devices that answered a broadcast on this gateway's networks.

Timeout (ms)

25000

Rescan

How long to wait for devices to answer. The gateway clamps this to its own bounds.

ICX Launcher Site
192.168.42.100

rest
Hostname ctrlX-OS.local

Connection URL

ctrlx:rest://192.168.42.100:443?tls.pinned-  
fingerprints=1d:3f:20:2b:21:9f:f5:62:c3:a3:d0:75:e1:08:3e:5b:2a:02:be:a1:3a:ef:63:ba:92:ac:15:43:fe:8e:e6:ae  
hostname  
ctrlX-OS.local  
instance-name  
ICX Launcher Site  
ip-address  
192.168.42.100  
ipv6-addresses  
fd40:e56f:3382:0:260:34ff:fe59:7615,2003:e0:373a:7e00:260:34ff:fe59:7615

Cancel

Apply

Selecting a result fills in **Host**, **Port** and **Pinned Certificate Fingerprints**. Four things it deliberately does not do:

- **It never fills Username or Password.** A device that answers a search tells you where it is, never who may log in to it. ctrlX refuses every connection that arrives without credentials, so these two are always yours to enter — a device you found is not a device you can read until you have.
- **It does not fill in the port the device advertised.** A ctrlX advertises its ICX Launcher web site, typically 8089, which is not the Data Layer this driver talks to. The port filled in is the Data Layer's 443.
- **It does not turn Verify SSL off.** A search never weakens a security setting on your behalf; it pins the exact certificate it saw instead — see below.
- **It fills in only what was actually observed.** A field the search learned nothing about keeps the value you gave it, and if applying a result would overwrite something you have already typed, the form lists what would change and asks first.

Create Device Connection

Create Device Connection

1. Select Driver

2. Configure Device

Hostname or IP address of the CtrlX device

Port

443

Port number (default: 443)

CTRLX

Username

Username for authentication with the CtrlX device. Required - leave it blank and Create Device Connection stays disabled.

Password

☒ None

☐ Embedded

☐ Referenced

Verify SSL

☐

Whether to verify SSL certificates (default: false)

Base Path

/automation/api/v2

Base path for the REST API (default: /automation/api/v2)

Request Timeout

5000

Back

Create Device Connection

Running a search needs permission to create device connections; being logged in is not enough.

## The fingerprint a search fills in

**Pinned Certificate Fingerprints** is filled with the SHA-256 of the certificate the search actually saw on that device. It is what makes a discovered ctrlX connectable at all: a factory certificate chains to no public CA, so without a pin the connection fails its TLS check against the very device that just answered.

Create Device Connection

Create Device Connection

1. Select Driver

2. Configure Device

Verify SSL

☐

Whether to verify SSL certificates (default: false)

Base Path

Base path for the REST API (default: /automation/api/v2)

Request Timeout

Default timeout for requests in milliseconds (default: 5000)

Pinned Certificate Fingerprints

Comma-separated SHA-256 fingerprints of the leaf certificates to accept (hex, colons optional). When set, only these certificates are accepted and Verify SSL is overridden - the secure way to reach a ctrlX device with its factory self-signed certificate, which chains to no public CA. The connect log prints the device's fingerprint when Verify SSL is off, ready to copy.

Browse Queries

Data Layer path to browse for symbols (default: "/plc/app/Application/sym/PLC\_PRG", lists all tags of a program)

LIMITS

Max Browse Depth

Maximum nesting depth a \*\* wildcard descends. Each level costs one HTTP browse per node, so an unexpectedly deep or

Back

Create Device Connection

Treat it as trust on first use. The fingerprint was read over a connection that could not yet be verified — which is the only way to read it the first time — so it is exactly as trustworthy as your network was at that moment. If you can compare it with the value the device's own web UI reports, do so. If you cannot, you still know precisely which certificate you are trusting, which is more than switching **Verify SSL** off would tell you.

## Setting blocks

The configuration is organized into categories: **General**, **Transport**, **CtrlX**, **Limits** and **Audit Log**.

### General

Assigns a name (and optional description) to the connection and controls whether it is enabled.

GENERAL

Description

An optional description for this device connection.

Enabled

☒ Set whether this device connection is enabled or disabled within the Gateway.

| Setting     | Description                              | Default |
|-------------|------------------------------------------|---------|
| Name        | Unique name for this device connection   |         |
| Description | Optional description for this connection |         |
| Enabled     | Whether the connection is active         | true    |

This is the name the connection is then referenced under in other parts of Ignition.

## Transport

TRANSPORT

Host

192.168.42.100

Hostname or IP address of the CtrlX device

Port

443

Port number (default: 443)

| Setting | Description                                | Default |
|---------|--------------------------------------------|---------|
| Host    | Hostname or IP address of the ctrlX device |         |
| Port    | HTTPS port of the Data Layer REST API      | 443     |

There is no transport selector, and none is missing: this driver speaks HTTPS and nothing else. The port is a plain override for a device behind a reverse proxy or a non-standard listener.



## CTRLX

### Username

Username for authentication with the CtrlX device

### Password

☐ None☒ Embedded☐ Update Password?

#### Secret

☐ Referenced

### Verify SSL

☐

Whether to verify SSL certificates (default: false)

### Base Path

Base path for the REST API (default: /automation/api/v2)

### Request Timeout

Default timeout for requests in milliseconds (default: 5000)

### Pinned Certificate Fingerprints

Comma-separated SHA-256 fingerprints of the leaf certificates to accept (hex, colons optional). When set, only these certificates are accepted and Verify SSL is overridden - the secure way to reach a ctrlX device with its factory self-signed certificate, which chains to no public CA. The connect log prints the device's fingerprint when Verify SSL is off, ready to copy.

### Browse Queries

Data Layer path to browse for symbols (default: "/plc/app/Application/sym/PLC\_PRG", lists all tags of a program)

| Setting                                | Description                                                                | Default                          |
|----------------------------------------|----------------------------------------------------------------------------|----------------------------------|
| <b>Username</b>                        | Account the driver authenticates as                                        |                                  |
| <b>Password</b>                        | That account's password                                                    |                                  |
| <b>Verify SSL</b>                      | Whether the device certificate is verified                                 | false                            |
| <b>Base Path</b>                       | Base path of the REST API                                                  | /automation/api/v2               |
| <b>Request Timeout</b>                 | Milliseconds allowed for one HTTP request                                  | 5000                             |
| <b>Pinned Certificate Fingerprints</b> | SHA-256 fingerprints of the exact leaf certificates to accept              |                                  |
| <b>Browse Queries</b>                  | Data Layer path(s) to browse for symbols; separate multiple queries with ; | /plc/app/Application/sym/PLC_PRG |

**Username and password are both required.** ctrlX authenticates every request and the driver refuses an unauthenticated connection. They are *not* part of the connection string: the driver takes them as a credential object, which is why they never appear in a logged URL.

**The password is held in Ignition's secret store.** The picker offers *None*, *Embedded* and *Referenced*: **Embedded** keeps the secret with this device, **Referenced** takes it from a configured secret provider, and *None* leaves the driver with no password — which this driver cannot connect with.

### Reaching a device with its factory certificate

A ctrlX CORE's factory certificate is self-signed, so it chains to no public CA and names the device rather than its address. Ordinary verification cannot pass, and since Connect 1.2.0 **Verify SSL** also checks the hostname — which such a certificate fails on both counts.

**Pin it.** Put the device's SHA-256 leaf fingerprint in **Pinned Certificate Fingerprints** (hex, colons optional; comma-separate several). A pin accepts exactly the certificate you listed and refuses a substituted one even though that one is equally self-signed, which is strictly stronger than turning verification off. Pinning **overrides Verify SSL**, and the hostname is deliberately not checked in that mode — the pin already identifies the peer.

To obtain the fingerprint, connect once with **Verify SSL** off: the driver logs the device's SHA-256 ready to copy. Then paste it into the field. Where your site runs its own CA, provisioning a real certificate remains the better answer.

**Verify SSL** off accepts any peer at all, which is why it is a starting point for getting the fingerprint and not a resting place.

## Limits

**LIMITS**

**Max Browse Depth**

Maximum nesting depth a \*\* wildcard descends. Each level costs one HTTP browse per node, so an unexpectedly deep or self-referential tree spends requests rather than stack. Real ctrlX Data Layer trees are a handful of levels deep.

| Setting                 | Description                    | Default |
|-------------------------|--------------------------------|---------|
| <b>Max Browse Depth</b> | How far a ** wildcard descends | 20      |

Each level a \*\* wildcard descends costs one HTTP browse per node, so an unexpectedly deep — or self-referential — subtree spends requests rather than stack. Real Data Layer trees are a handful of levels deep, and the default of 20 is far above anything legitimate; it is a backstop, not a tuning knob.

## Audit Log

**AUDIT LOG**

**Audit-Log File Path**

Path to where the audit-log file will be written. Leave empty to disable audit logging.

| Setting                    | Description                                                         | Default |
|----------------------------|---------------------------------------------------------------------|---------|
| <b>Audit-Log File Path</b> | Path to a file protocol audit events are written to; empty disables |         |

The audit log records protocol-level events and grows quickly, so enable it only while investigating a problem.

## Tag Addressing

---

ctrlX addresses are Data Layer node paths — slash-separated, like a filesystem, **with no leading slash**:

```
<segment>(/<segment>)*
```

| Address                                          | Meaning                                                      |
|--------------------------------------------------|--------------------------------------------------------------|
| plc/app/Application/sym/PLC_PRG/myVariable       | A PLC variable exposed through the Data Layer                |
| motion/axs/MyAxis/state/values/actual/pos        | An axis's actual position                                    |
| framework/metrics/system/cpu-utilisation-percent | A single CPU-utilisation node                                |
| framework/metrics/system/**                      | Pattern — every descendant of that namespace (browsing only) |

A leading `/` or stray whitespace is the usual cause of `INVALID_ADDRESS` from a read or write. Note the asymmetry with **Browse Queries**, whose default *does* begin with `/`: that field is a browse path, not a tag address.

The Data Layer tree is large. Restrict **Browse Queries** to the subtree you actually want — the default, `/plc/app/Application/sym/PLC_PRG`, lists the symbols of one PLC program — rather than browsing from the root.

## Types

Data Layer nodes carry their type in the response payload (`int8`, `uint16`, `float64`, `string`, `bool`, ...), and the driver maps that to the matching OPC UA type. Nothing has to be declared in the address.

A node the controller reports without a usable type is skipped, with a `WARN` naming it, and the rest of the browse is unaffected — one untypeable node does not cost the device.

## Tag Groups

---

By default, all tags are collected in the default tag group, which means all selected tags are fetched at the default interval of once a second.

If different intervals are desired, create additional tag groups, configure the desired intervals there, and assign tags to the corresponding tag groups.

**This matters more here than on a fieldbus driver.** Each tag in a read is a separate HTTP request, so a hundred tags at one second is a hundred requests a second against a management interface. Put slow-moving diagnostics in a slower group.

## Troubleshooting

---

### Device shows "Faulted" or sits in "Reconnecting":

- Verify the device is reachable over HTTPS on the configured port — `curl -k https://<host>/` is enough to tell a network problem from a certificate one
- Review Gateway logs: **Status > Diagnostics > Logs**

### SSL handshake failed although the device answers:

- The factory certificate cannot pass ordinary verification. Pin its fingerprint under **Pinned Certificate Fingerprints** — see *Reaching a device with its factory certificate*

### HTTP 401 Unauthorized:

- Wrong username or password, or the account was left empty

### HTTP 403 Forbidden — reads work, writes are refused:

- The account authenticated, but lacks the role the operation needs. ctrlX manages read and write as separate permissions; adjust the role in the controller's user management

### HTTP 404 Not Found:

- The node path does not exist. Browse to discover legal paths rather than typing them

### INVALID\_ADDRESS from a read or write:

- Almost always a leading `/` or whitespace in the address. Tag addresses have no leading slash

### Bulk reads are slow:

- Expected: each tag issues its own HTTP call. Spread tags across slower tag groups

## Docker related issues

The ctrlX protocol carries no identity for this end of the connection, so a container's internal address is never announced to the device and nothing has to be reconciled. A gateway in a bridged container connects normally, provided the device's HTTPS port is routable.

**Device discovery is the exception.** *Discover Devices* searches with mDNS, which is multicast: a bridged container sits on the Docker-internal network, so the query never reaches the network the controllers are on and nothing answers. Enter the host by hand — everything else on the page works normally. Host networking ( `--network host` , Linux only) puts the gateway on the plant network, and the search then works.

## VPN related issues

A remote device reached over a VPN needs nothing special from this driver, for the same reason: the connection is an ordinary outbound HTTPS session. Only reachability and the certificate matter.

## Version history

---

### Unreleased — 1.3.0

- **Added: Discover Devices** on Ignition 8.3 — the device page can now search the gateway's networks for ctrlX controllers and list the ones that confirm themselves, so the address no longer has to be typed from a commissioning note. Picking a result fills in the host, the Data Layer port and the pinned certificate fingerprint of the certificate that device actually presented; it never fills a username or password, and never turns **Verify SSL** off. Requires permission to create device connections, and does not work from inside a bridged container.
- First release of this manual.